Федеральное государственное образовательное бюджетное
учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики» (ФГОБУ ВО «СибГУТИ»)

Основы систем мобильной связи

Лабораторная работа 1

Выполнил: Клишин Владиммир Евгеньевич ИКС - 431

Новосибирск 2026

Цель работы:
Изучить формы радиосигналов, их частотно-временное представление, преобразования Фурье, основы АЦП, выбор частоты дискретизации и влияние разрядности квантования на спектр.

1) Необходимо сгенерировать и визуализировать (вывести на график) непрерывный сигнал, в соответствии с вариантом (таблица 1.1). Номер варианта – порядковый номер в журнале группы.

11	$y(t) = 8\sin\left(2\pi f t + \frac{\pi}{3}\right), \quad f = 18$

Код:

```
import numpy as np
import matplotlib.pyplot as plt

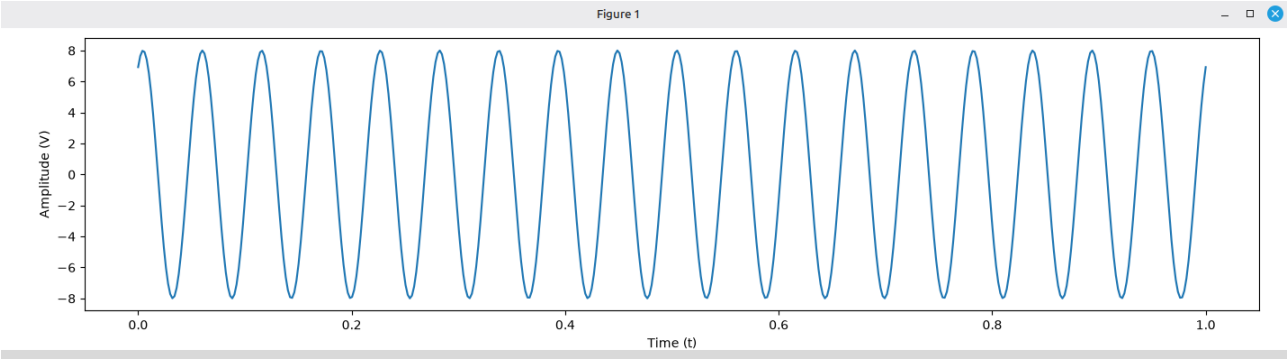
A = 8
f = 18

t = np.linspace(0, 1, 500)
phi = 60

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)

plt.subplot(1, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')
plt.tight_layout()
plt.show()
```

График:



2) Определить максимальную частоту в спектре данного сигнала.

Максимальная частота равна частоте сигнала потому, что у меня в варианте чистый синус

Код:

```
import numpy as np
import matplotlib.pyplot as plt

A = 8
f = 18

f_max = f
print("Максимальная частота:", f_max)


t = np.linspace(0, 1, 500)
phi = 60

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)

plt.subplot(1, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')
plt.tight_layout()
plt.show()
```

Вывод:

```
Максимальная частота: 18
```

3)Определить минимальную необходимую частоту дискретизации полученного сигнала (теорема Котельникова)

Код:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
A = 8
f = 18

f_max = f
print("Максимальная частота:", f_max)

N = 2 * f_max
print("Минимальная частота дискретизации:", N)

t = np.linspace(0, 1, 500)
phi = 60

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)

plt.subplot(1, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')
plt.tight_layout()


plt.show()
```

Вывод:

```
Максимальная частота: 18
Минимальная частота дискретизации: 36
```

4)Оцифровать сигнал с полученной частотой дискретизации, выбрав требуемое число отсчетов сигнала на длительности 1 секунда, сохранить полученные значения в массив, пока не озадачиваясь разрядностью АЦП, просто выбранные с частотой дискретизации значения с теми уровнями, которые имеет функция в полученных точках.

Код:

```
import numpy as np
import matplotlib.pyplot as plt

A = 8
f = 18

f_max = f
print("Максимальная частота:", f_max)

N = 2 * f_max
print("Минимальная частота дискретизации:", N)

t = np.linspace(0, 1, N, endpoint=False)
```

```
t2 = np.linspace(0, 1, 500)
phi = 60

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)
x2 = A * np.sin(2 * np.pi * f * t2 + phi * np.pi / 180)

plt.subplot(2, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.subplot(2, 1, 2)
plt.plot(t2, x2)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')


plt.tight_layout()
plt.show()
```

5)Выполнить прямое дискретное преобразование Фурье для массива временных отсчетов сигнала и оценить ширину данного спектра. А также объем памяти, требуемый для хранения данного массива (тип переменных в
массиве – на ваше усмотрение, float, int и пр.).

Код:

```
import numpy as np
import matplotlib.pyplot as plt

A = 8
f = 18
f_max = f
N = 2 * f_max
fs = N / 1
delta_f = fs / N

freqs = np.arange(N) * (fs / N)
t = np.linspace(0, 1, N, endpoint=False)
t2 = np.linspace(0, 1, 500)
phi = 60



print("Максимальная частота:", f_max)
print("Минимальная частота дискретизации:", N)

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)
```

```
x2 = A * np.sin(2 * np.pi * f * t2 + phi * np.pi / 180)

fur = [0] * N

#X[k] = Σ (n=0..N-1) x[n] * exp(-j * 2π * k * n / N)

for k in range(N):
for n in range(N):
fur[k] += x[n] * np.exp(-1j * 2 * np.pi * k * n / N)

absComp = np.abs(fur)

porog = 0.05 * np.max(absComp)


bins = []
for i in range((len(absComp) // 2) + 1):
if absComp[i] > porog:
bins.append(i)

f_low = freqs[bins[0]]
f_high = freqs[bins[-1]]
spectral_width = max(f_high - f_low, delta_f)

print("Ширина спектра:", spectral_width)

print("Память массива отсчетов(там float64): ", len(x) * 8, "байт")
print("Память массива сектра(массив комплексных чисел complex128):", len(fur) * 16, "байт")


plt.subplot(2, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.subplot(2, 1, 2)
plt.plot(t2, x2)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

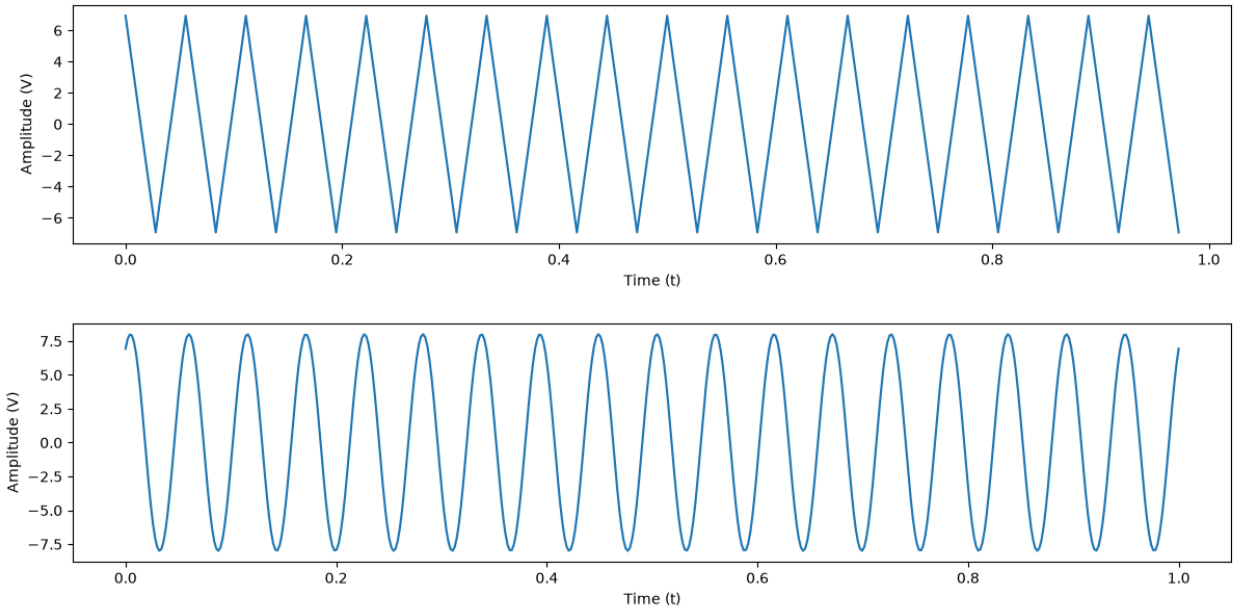
plt.tight_layout()
plt.show()
```

Вывод:

```
Максимальная частота: 18
Минимальная частота дискретизации: 36
Ширина спектра: 1.0
Память массива отсчетов(там float64):  288 байт
Память массива сектра(массив комплексных чисел complex128): 576 байт
```

6)Восстановите оригинальный аналоговый сигнал по массиву имеющихся у вас отсчетов, соединив их непрерывной линией и оцените визуальное сходство оригинального сигнала и восстановленного после оцифровки.

График:



7)Увеличьте частоту дискретизации в 4 раза и проделайте задания из п.4-6.

Код:

```
import numpy as np
import matplotlib.pyplot as plt

A = 8
f = 18
f_max = f
N = 144
fs = 144
delta_f = fs / N

freqs = np.arange(N) * (fs / N)
t = np.linspace(0, 1, N, endpoint=False)
t2 = np.linspace(0, 1, 500)
phi = 60
```

```
print("Максимальная частота:", f_max)
print("Минимальная частота дискретизации:", 2 * f_max)

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)
x2 = A * np.sin(2 * np.pi * f * t2 + phi * np.pi / 180)

fur = [0] * N

#X[k] = Σ (n=0..N-1) x[n] * exp(-j * 2π * k * n / N)

for k in range(N):
for n in range(N):
fur[k] += x[n] * np.exp(-1j * 2 * np.pi * k * n / N)

absComp = np.abs(fur)

porog = 0.05 * np.max(absComp)


bins = []
for i in range((len(absComp) // 2) + 1):
if absComp[i] > porog:
bins.append(i)

f_low = freqs[bins[0]]
f_high = freqs[bins[-1]]
spectral_width = max(f_high - f_low, delta_f)

print("Ширина спектра:", spectral_width)

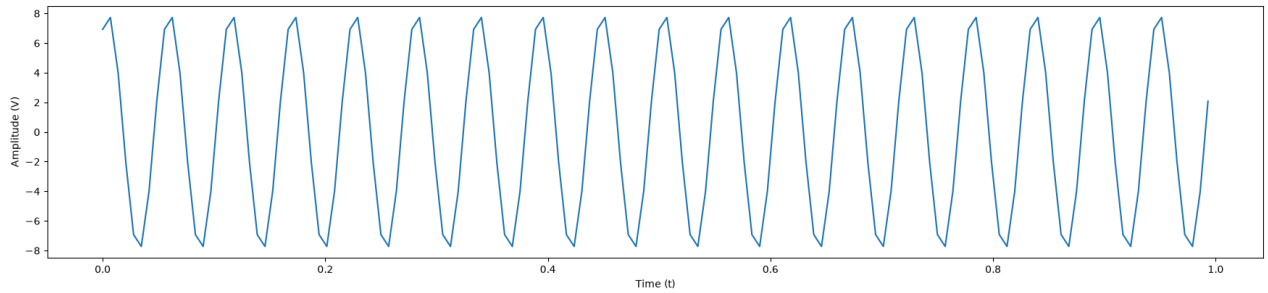
print("Память массива отсчетов(там float64): ", len(x) * 8, "байт")
print("Память массива сектра(массив комплексных чисел complex128):", len(fur) * 16, "байт")

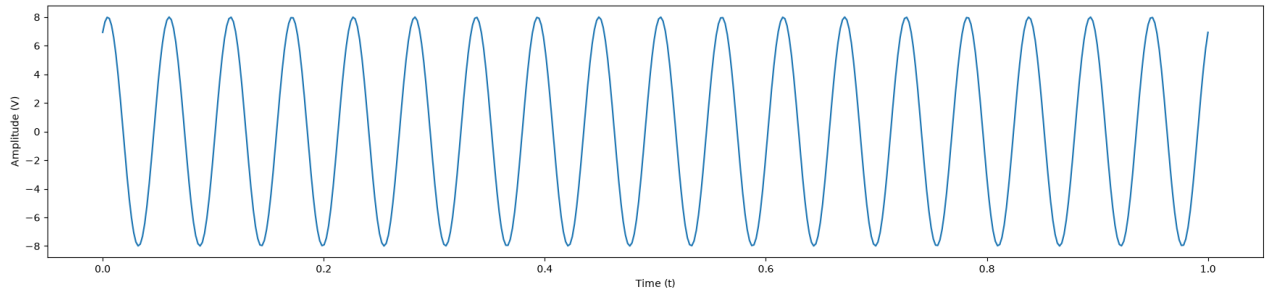

plt.subplot(2, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.subplot(2, 1, 2)
plt.plot(t2, x2)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.tight_layout()
plt.show()
```

График:

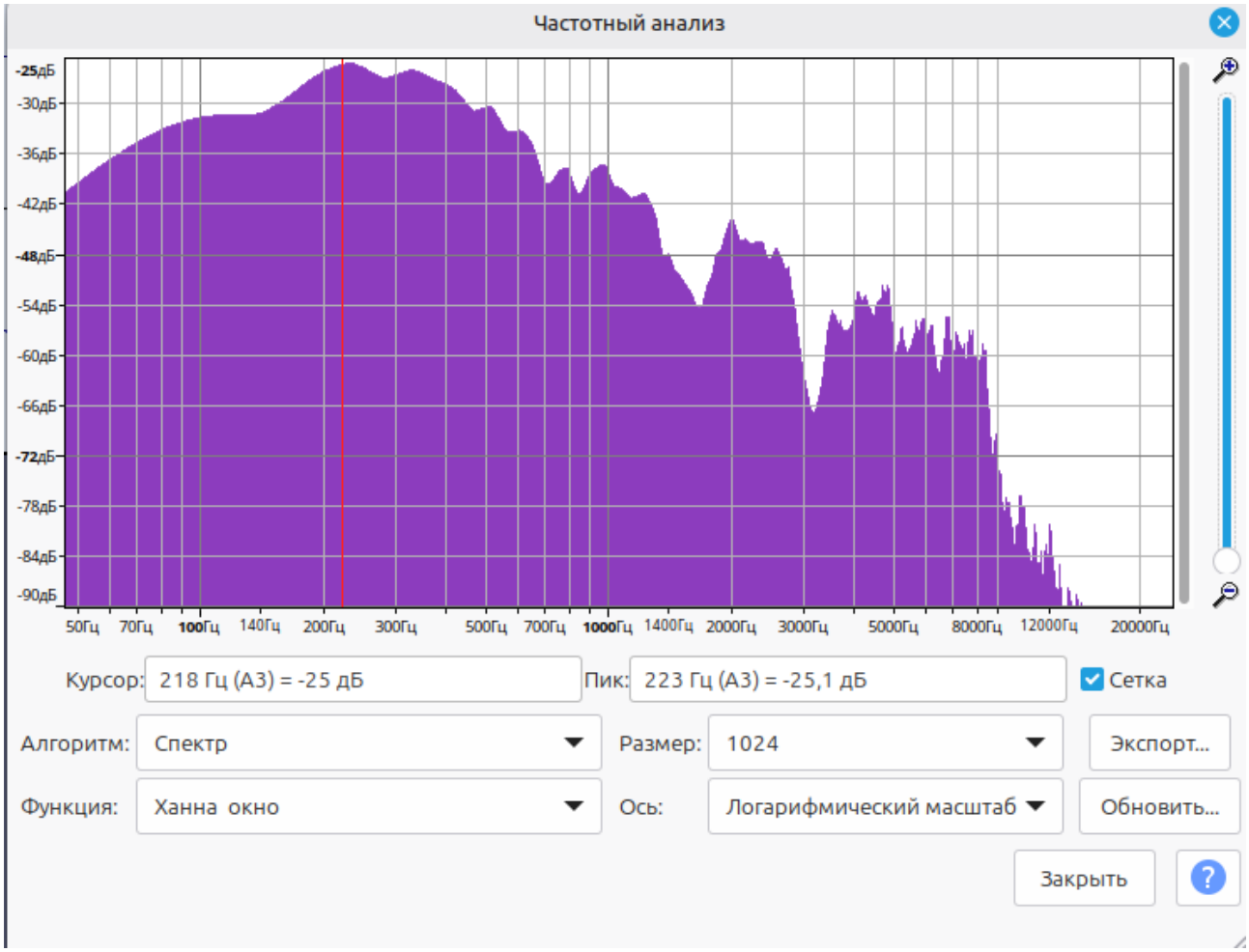




Вывод:

Максимальная частота: 18
Минимальная частота дискретизации: 36
Ширина спектра: 1.0
Память массива отсчетов(там float64): 1152 байт
Память массива сектра(массив комплексных чисел complex128): 2304 байт

8) Запишите аудиофайл со своим голосом (например, в формате wav). Проанализируйте визуально спектр голоса, используя, например, приложение
Audacity (вкладки Анализ -> График спектра). Определите максимальную частоту в спектре данного сигнала и выберите требуемую для оцифровки частоту дискретизации.



9)Используя библиотеки работы со звуком Matlab (или Pyton) проанализируйте имеющуюся у вас запись голоса. Считайте аудиофайл с помощью функции Matlab (файл с голосом должен лежать в рабочей директории Matlab)

```
[y, Fs] = audioread('golos.wav');


fprintf('Частота дискретизации: %d Гц\n', Fs);
```

10) Определите частоту дискретизации, которая была использована при записи голоса на цифровой носитель. Для этого возьмите количество элементов в файле с записью и разделите на длительность записи в секундах.
Например, если у вас файл состоит из 441 000 элементов (отсчетов) и имеет длительность 10 секунд, то частота дискретизации Fs будет равна 441 000/10 =
44 100 Гц. Сравните вычисленное значение с тем, что выводится в Matlab для

Fs.

Код:

```
[y, Fs] = audioread('golos.wav');


fprintf('Частота дискретизации: %d Гц\n', Fs);

y = y(:, 1);

dlitelnost = length(y) / Fs;
fprintf('Длительность записи: %.2f секунд\n', dlitelnost);

Fs_calc = length(y) / dlitelnost;
fprintf('Вычисленная частота дискретизации: %d Гц\n', Fs_calc);
```

Вывод:

```
Частота дискретизации: 48000 Гц
Длительность записи: 4.27 секунд
Вычисленная частота дискретизации: 48000 Гц
```

11) Проредите полученный массив y с помощью функции downsample (иными словами – уменьшите частоту дискретизации) и воспроизведите полученный сигнал с помощью matlab-функции play() или ее аналогами в других языках программирования. Обратите внимание на качество звучания при неверно взятой частоте дискретизации

Код:

```
[y, Fs] = audioread('golos.wav');


fprintf('Частота дискретизации: %d Гц\n', Fs);

y = y(:, 1);

dlitelnost = length(y) / Fs;
fprintf('Длительность записи: %.2f секунд\n', dlitelnost);

Fs_calc = length(y) / dlitelnost;
fprintf('Вычисленная частота дискретизации: %d Гц\n', Fs_calc);

%проредили
```

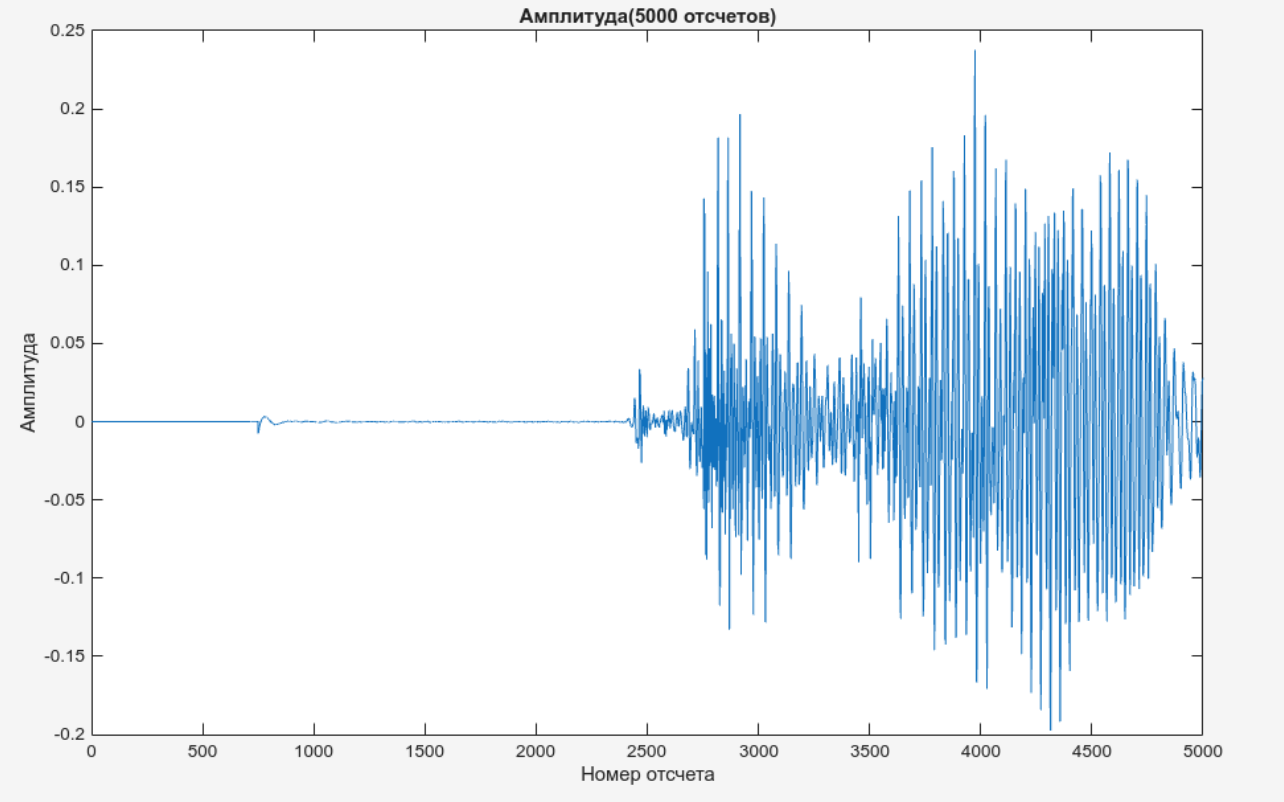
```
y_down = downsample(y, 10);

fprintf('Новая частота дискретизации: %d Гц\n', Fs / 10);

zvuk = audioplayer(y_down, Fs / 10);
disp('Чтобы остановить введи stop(zvuk) в командное окно)');
play(zvuk);
pause(4);

figure;
plot(y_down(1: 5000));
title('Амплитуда(5000 отсчетов)');
xlabel('Номер отсчета');
ylabel('Амплитуда');
```

График:



12) Выполните прямое дискретное преобразование Фурье для оригинального звучания и для прореженного сигнала, выведите на график амплитудный спектр сигнала, определите его ширину.

Код:

```
[y, Fs] = audioread('golos.wav');
```

```
fprintf('Частота дискретизации: %d Гц\n', Fs);

y = y(:, 1);

dlitelnost = length(y) / Fs;
fprintf('Длительность записи: %.2f секунд\n', dlitelnost);

Fs_calc = length(y) / dlitelnost;
fprintf('Вычисленная частота дискретизации: %d Гц\n', Fs_calc);

%проредили
y_down = downsample(y, 10);

fprintf('Новая частота дискретизации: %d Гц\n', Fs / 10);

zvuk = audioplayer(y_down, Fs / 10);
disp('Чтобы остановить введи stop(zvuk) в командное окно)');
play(zvuk);
pause(4);

figure;
plot(y_down(1: 5000));
title('Амплитуда(5000 отсчетов)');
xlabel('Номер отсчета');
ylabel('Амплитуда');

%% 12


N = 1024;
x_orig = y(1:N * 10);
x_down = y_down(1:N);

X_orig = zeros(1, N * 10);
X_down = zeros(1, N);


for k = 1:(N * 10)
    sum_val = 0;
    for n = 1:(N * 10)
        sum_val = sum_val + x_orig(n) * exp(-1j * 2 * pi * (k-1) * (n-1) / N);
    end
    X_orig(k) = sum_val;
end


for k = 1:N
    sum_val = 0;
    for n = 1:N
        sum_val = sum_val + x_down(n) * exp(-1j * 2 * pi * (k-1) * (n-1) / N);
    end
    X_down(k) = sum_val;
```

```
end


spectrum_orig = abs(X_orig);
spectrum_down = abs(X_down);


f_orig = (0:N-1) * (Fs / N);

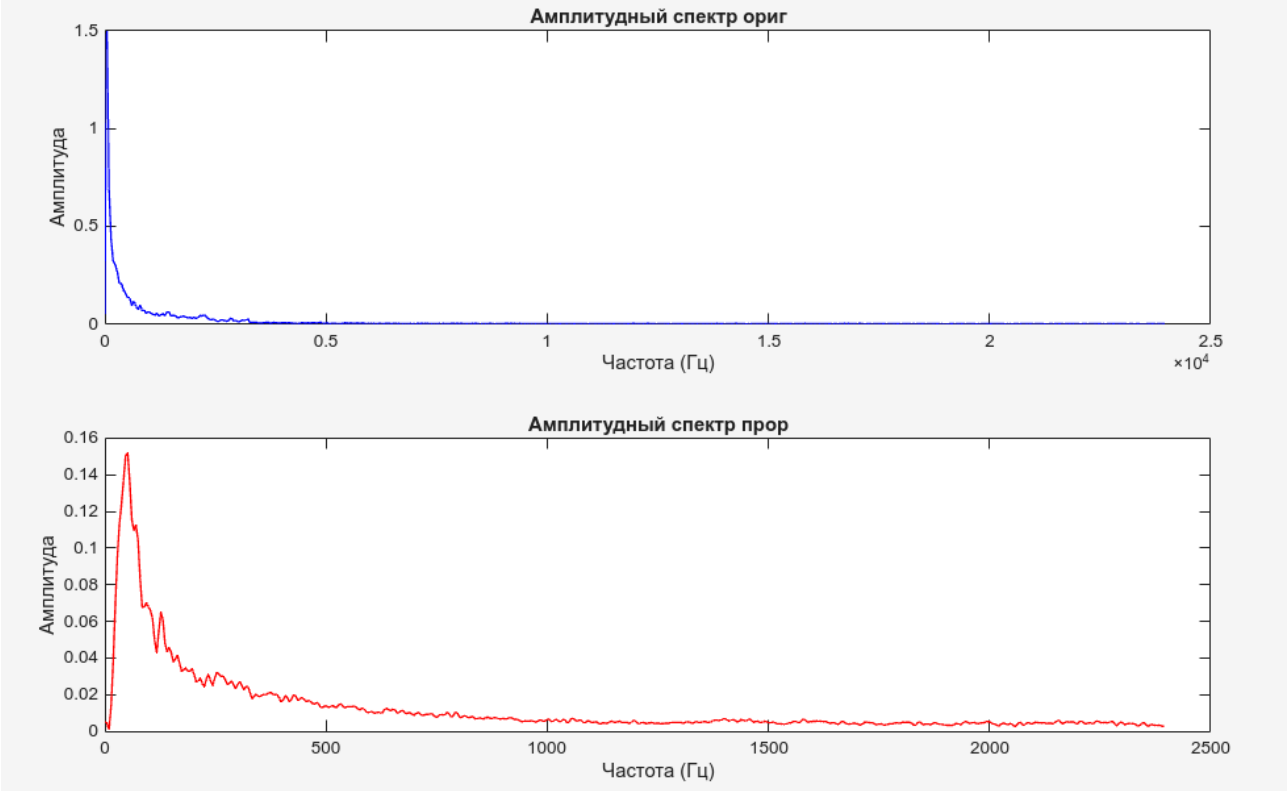
f_down = (0:N-1) * ((Fs / 10) / N);

figure;

subplot(2,1,1);
plot(f_orig(1:N/2), spectrum_orig(1:N/2), 'b', 'LineWidth', 1);
title('Амплитудный спектр ориг');
xlabel('Частота (Гц)');
ylabel('Амплитуда');

subplot(2,1,2);
plot(f_down(1:N/2), spectrum_down(1:N/2), 'r', 'LineWidth', 1);
title('Амплитудный спектр прор');
xlabel('Частота (Гц)');
ylabel('Амплитуда');
```

График:



13) Оцените влияние разрядности АЦП на спектр сигнала. Для этого

нужно написать функцию, которая бы округляла значения отсчетов сигнала,
заданного в вашем варианте, до какого-то числа, определяемого разрядностью
АЦП. Допустим если у АЦП всего 3 разряда, то диапазон возможных дискретных значений амплитуд временных отсчетов сигнала – это 0..7 (то есть,
7=23-1). Все значения больше 7 округляются до 7. Для результирующего дискретного сигнала требуется выполнить прямое преобразование Фурье. Сравнить полученный спектр со спектром исходной синусоиды, отсчеты которой не подвергались квантованию по уровню. Вывести среднюю ошибку
квантования для случаев, когда разрядность АЦП равна 3/4/5/6.

Код:

```
import numpy as np
import matplotlib.pyplot as plt

A = 8
f = 18
f_max = f
N = 2 * f_max
fs = N / 1
delta_f = fs / N

freqs = np.arange(N) * (fs / N)
t = np.linspace(0, 1, N, endpoint=False)
t2 = np.linspace(0, 1, 500)
phi = 60


print("Максимальная частота:", f_max)
print("Минимальная частота дискретизации:", N)

x = A * np.sin(2 * np.pi * f * t + phi * np.pi / 180)
x2 = A * np.sin(2 * np.pi * f * t2 + phi * np.pi / 180)

fur = [0] * N

#X[k] = Σ (n=0..N-1) x[n] * exp(-j * 2π * k * n / N)

for k in range(N):
for n in range(N):
fur[k] += x[n] * np.exp(-1j * 2 * np.pi * k * n / N)

absComp = np.abs(fur)

porog = 0.05 * np.max(absComp)
```

```
bins = []
for i in range((len(absComp) // 2) + 1):
if absComp[i] > porog:
bins.append(i)

f_low = freqs[bins[0]]
f_high = freqs[bins[-1]]
spectral_width = max(f_high - f_low, delta_f)

print("Ширина спектра:", spectral_width)

print("Память массива отсчетов(там float64): ", len(x) * 8, "байт")
print("Память массива сектра(массив комплексных чисел complex128):", len(fur) * 16, "байт")


plt.subplot(2, 1, 1)
plt.plot(t, x)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.subplot(2, 1, 2)
plt.plot(t2, x2)
plt.xlabel('Time (t)')
plt.ylabel('Amplitude (V)')

plt.tight_layout()
plt.show()


def quantize(signal, bits):

levels = 2 ** bits

signal_adc = (signal + A) / (2 * A) * (levels - 1)

signal_adc = np.round(signal_adc)
signal_adc = np.clip(signal_adc, 0, levels - 1)

signal_quant = signal_adc / (levels - 1) * (2 * A) - A

return signal_quant


def quantization_error(original, quantized):

error = np.abs(original - quantized)

return np.mean(error)
```

```
abs_original = np.abs(fur)


for bits in [3, 4, 5, 6]:

# Квантованный сигнал
x_quant = quantize(x, bits)

# Ошибка квантования
error = quantization_error(x, x_quant)

print("Разрядность АЦП:", bits, "бит")
print("Средняя ошибка квантования:", error)
print()


for i, bits in enumerate([3, 4, 5, 6]):

x_quant = quantize(x, bits)

fur_quant = [0] * N

for k in range(N):
for n in range(N):
fur_quant[k] += x_quant[n] * np.exp(-1j * 2 * np.pi * k * n / N)

abs_quant = np.abs(fur_quant)

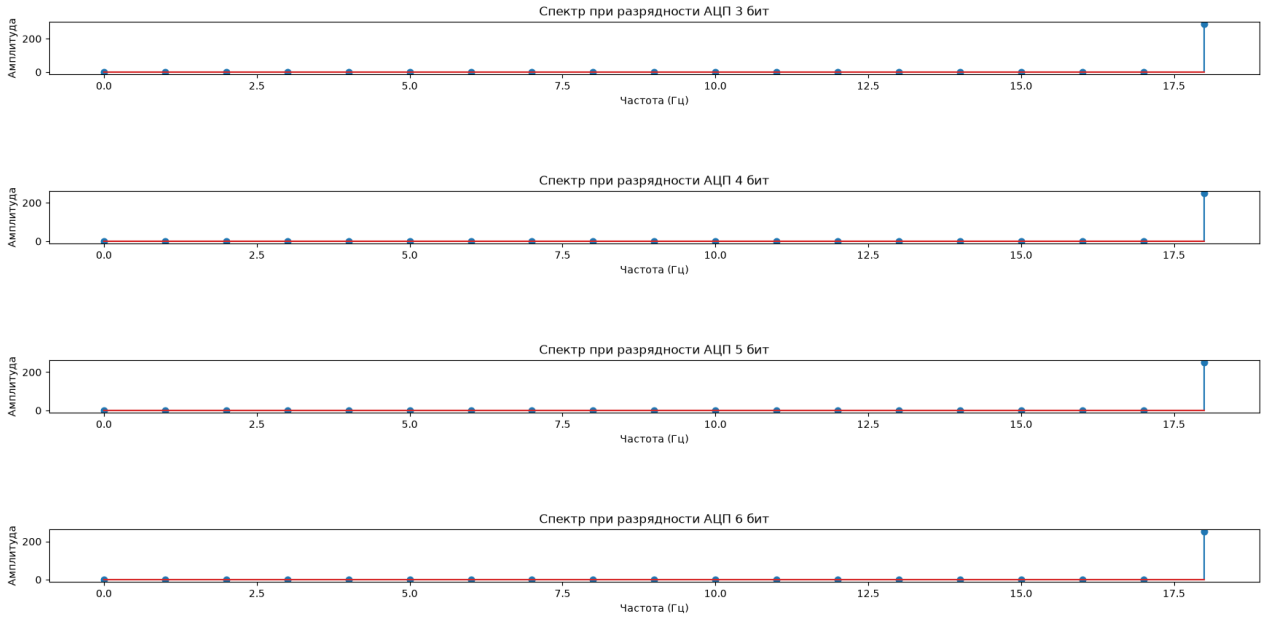
plt.subplot(4, 1, i + 1)

plt.stem(freqs[:N // 2 + 1], abs_quant[:N // 2 + 1])

plt.title("Спектр при разрядности АЦП " + str(bits) + " бит")
plt.xlabel("Частота (Гц)")
plt.ylabel("Амплитуда")

plt.tight_layout()
plt.show()
```

График:



Вывод:

Максимальная частота: 18
Минимальная частота дискретизации: 36
Ширина спектра: 1.0
Память массива отсчетов(там float64): 288 байт
Память массива сектра(массив комплексных чисел complex128): 576 байт
Разрядность АЦП: 3 бит
Средняя ошибка квантования: 1.0717967697245037

Разрядность АЦП: 4 бит
Средняя ошибка квантования: 0.005130103057837158

Разрядность АЦП: 5 бит
Средняя ошибка квантования: 0.03953870520837414

Разрядность АЦП: 6 бит
Средняя ошибка квантования: 0.05592375385148777

Вывод:
В ходе работы изучены основы оцифровки сигналов: подтверждена теорема Котельникова ($f_s \geq 2 \cdot f_{max}$), показано, что увеличение частоты дискретизации улучшает качество восстановления, но требует больше памяти. На примере записи голоса продемонстрировано влияние прореживания (алиасинг) и разрядности АЦП на спектр сигнала

Ссылка на Git:

Ответы на контрольные вопросы:

1) Для чего используются прямое и обратное преобразование Фурье?
2) Что такое ошибка квантования и дискретизации?
3) Какое количество разрядов АЦП требуется, чтобы оцифровать голос?
16
4) Как математически получить дискретные отсчеты непрерывного сигнала?
5) Какой спектр у периодического сигнала $\sin(10\pi t+\pi/2)$?
6) Что такое быстрое преобразование Фурье?
7) Как определяется минимальная требуемая для оцифровки частота дискретизации сигнала?

1)
Прямое - переводит сигнал из времени в частоту (получить спектр). Обратное - из частоты обратно во время (восстановить сигнал по спектру).

2)
Квантование - разница между реальным значением и округлённым до уровня (шум квантования). Дискретизация - замена непрерывного сигнала отсчётами через интервал; ошибка - потеря информации между отсчётами (наложение спектров, если частота мала).

3)
Обычно 16 бит (диапазон ~96 дБ, стандарт для речи/аудио). Минимум - 12–14 бит.

4)
Умножением непрерывного сигнала на гребёнку Дирака:
$x_d[n] = x(nT)$, где $T = 1/f_д$ - период дискретизации.

5)
Одна гармоника на частоте $f = 5$ Гц (т.к. $\omega = 10\pi$). Спектр - две дельта-функции на ±5 Гц (или одна линия на 5 Гц в одностороннем виде). Амплитуда 1, фаза $\pi/2$.

6)
Алгоритм, вычисляющий дискретное преобразование Фурье за $N \cdot \log_2 N$ операций вместо N^2. Просто ускоренный способ посчитать спектр.

7)
По теореме Котельникова: $f_д \geq 2 \cdot f_{max}$, где f_{max} — максимальная частота в спектре сигнала.